

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Haresh Kumar N L	Reg. No.:	192425009
Section / Batch:	AI&ML	Date:	06/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Q1) Big Data Sorting Framework (Merge vs Quick Sort)

A big data platform must process and sort terabytes of information efficiently. Analyze the use of Merge Sort and Quick Sort in this environment. Identify issues such as memory usage, scalability, and speed. Apply divide and conquer concepts to compare both algorithms.

Analyze performance under different conditions. Design a solution and justify the most suitable algorithm.

Code of Conduct

I Haresh Kumar N L (192425009) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Answer:

Big data platforms process enormous amounts of information, often reaching terabytes or petabytes of data. Efficient sorting is a critical operation because it improves data retrieval, searching, indexing, and analytics. In such environments, the choice of sorting algorithm significantly affects performance, memory usage, and scalability. Merge Sort and Quick Sort are two widely used Divide and Conquer algorithms that solve the sorting problem by recursively dividing the dataset into smaller parts and combining the results. This case study analyzes both algorithms and identifies the most suitable one for large-scale data processing.

Key Issues in Big Data Sorting:

1. Memory Usage:

Memory is a major concern when sorting terabytes of data. Merge Sort requires additional memory to merge subarrays, whereas Quick Sort performs sorting in-place and requires much less extra memory. However, when datasets are larger than the available RAM, Merge Sort is more suitable because it efficiently works with external storage.

2. Processing Speed:

Quick Sort usually provides faster execution for in-memory datasets because of its cache-friendly nature. Merge Sort has a consistent execution time but involves additional copying of data during the merge process.

3. Scalability:

Modern big data systems distribute data across multiple machines. Merge Sort can easily divide datasets among processors and merge the sorted results, making it highly scalable. Quick Sort is more difficult to parallelize efficiently because of its partitioning process.

4. Worst-Case Performance:

Quick Sort has a worst-case time complexity of $O(n^2)$ when poor pivot elements are selected. Merge Sort consistently performs in $O(n \log n)$ regardless of the input, making it more reliable for large datasets.

Analysis of Merge Sort and Quick Sort:

Merge Sort:

Merge Sort divides the dataset into two equal halves, recursively sorts each half, and merges them into a sorted sequence. Its performance remains $O(n \log n)$ for all cases, making it highly predictable. Because of its stability and suitability for external sorting, it is widely used in distributed computing environments.

Quick Sort:

Quick Sort selects a pivot element and partitions the dataset into smaller and larger values. It generally performs faster than Merge Sort for in-memory applications due to lower overhead. However, its efficiency depends on selecting a good pivot. Poor pivot selection can significantly reduce performance.

Performance Comparison:

Feature	Merge Sort	Quick Sort
Algorithm Technique	Divide and Conquer	Divide and Conquer
Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n \log n)$	$O(n^2)$
Extra Memory	$O(n)$	$O(\log n)$
Stability	Stable	Not Stable

Feature	Merge Sort	Quick Sort
Scalability	Excellent	Good
External Sorting	Excellent	Limited

Proposed Solution:

For a big data sorting framework, Merge Sort is the most appropriate algorithm because it provides guaranteed performance, supports distributed computing, and efficiently handles datasets larger than the available memory. Although Quick Sort performs better for smaller datasets stored entirely in RAM, Merge Sort is more reliable for enterprise-scale applications involving terabytes of data. Therefore, the system should adopt Merge Sort for large-scale data processing while using Quick Sort only for smaller in-memory datasets.

Justification:

The proposed solution ensures consistent execution time, supports parallel processing, and minimizes the risk of performance degradation. In distributed platforms such as Hadoop and Spark, Merge Sort integrates naturally with the MapReduce programming model, where individual nodes sort partitions independently before merging the final output. This makes Merge Sort the preferred algorithm for big data applications despite its higher memory requirements.

A big data platform must process and sort terabytes of information efficiently. Analyze the use of Merge Sort and Quick Sort in this environment. Identify issues such as memory usage, scalability, and speed. Apply divide and conquer concepts to compare both algorithms. Analyze performance under different conditions. Design a solution and justify the most suitable algorithm.

Your Code

```
1 import random
2 import time
3 def merge_sort(arr):
4     if len(arr)<=1:
5         return arr
6     mid=len(arr)//2
7     left=merge_sort(arr[:mid])
8     right=merge_sort(arr[mid:])
9     return merge(left,right)
10 def merge(left,right):
11     result=[]
12     i=j=0
13     while i<len(left) and j<len(right):
14         if left[i]<=right[j]:
15             result.append(left[i])
16             i+=1
17         else:
18             result.append(right[j])
19             j+=1
20     result.extend(left[i:])
21     result.extend(right[j:])
22     return result
23 def quick_sort(arr):
24     if len(arr)<=1:
25         return arr
26     pivot=arr[len(arr)//2]
27     left=[x for x in arr if x<pivot]
28     middle=[x for x in arr if x==pivot]
29     right=[x for x in arr if x>pivot]
30     return quick_sort(left)+middle+quick_sort(right)
31 n=int(input("Enter Number of Element:"))
32 data=[random.randint(1,1000000)for _ in range(n)]
33 merge_data=data.copy()
34 quick_data=data.copy()
35 start=time.perf_counter()
36 merge_result=merge_sort(merge_data)
37 merge_time=time.perf_counter()-start
38 quick_result=quick_sort(quick_data)
39 quick_time=time.perf_counter()-start
40 print("\nMerge Sort Time:",round(merge_time,6),"Seconds")
41 print("Quick Sort Time:",round(quick_time,6),"Seconds")
42 print("\nSorting Correct:",merge_result==quick_result)
43 if merge_time<quick_time:
44     print("Recommendation: Merge Sort performed better.")
45 elif quick_time<merge_time:
46     print("Recommendation: Quick Sort performed better.")
47 else:
48     print("Both algorithms Showed Similar performance.")
```

Input

Output

```
Enter Number of Element:
Merge Sort Time: 0.043091
Seconds
Quick Sort Time: 0.079501
Seconds
```

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____